AUTOMATA-BASED URL VALIDATOR

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102241	**A.VISHNU VARDHAN**
111624102242	**VISHWAJITH R**
111624102246	**YOGESH S**
111624102247	**YUVARAJ U**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **A.VISHNU VARDHAN, VISHWAJITH R, YOGESH S and YUVARAJ U**, bearing **Register Numbers 111624102241, 111624102242, 111624102246 and 111624102247** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"Automata-Based URL Validator"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student **Project Guide**

A.VISHNU VARDHAN

VISHWAJITH R

YOGESH S

YUVARAJ U

ABSTRACT

A URL looks like free text, but it follows strict rules: a scheme, a host made of labels, an optional port, and an optional path, query and fragment. Programs usually check these rules either with a parsing library, which accepts many malformed addresses, or with a hand-written regular expression, which answers only yes or no. This mini project builds a URL validator from the foundations of the Theory of Computation. The URL syntax is written as one regular expression, with the numeric limits included: IPv4 parts up to 255, ports from 1 to 65535 and host labels of at most 63 characters. A compiler written from the textbook algorithms, using only the Python standard library, parses the expression, builds a non-deterministic finite automaton by Thompson's construction, converts it to a deterministic automaton by the subset construction, and reduces it with Moore's partition refinement. For the full URL grammar a 618-state NFA becomes a 299-state DFA and finally a minimal DFA of 258 states, built in about 60 milliseconds. Every URL is read once, one table look-up per character. When it fails, the validator reports the position of the first impossible character, the characters that would have been accepted, and whether the input is wrong or only unfinished; for valid URLs it splits the scheme, host, port, path, query and fragment. A command-line program and a browser page with a state-by-state trace expose the automaton. The validator agrees with all 36 labelled test cases and with Python's re module on 3,000 generated strings, the numeric ranges are verified exhaustively, and 21 automated tests pass. A library parsing check accepted 21 of the 24 invalid test URLs. The work shows concretely that a language with arithmetic-looking rules is still regular, and that the price of precision is the number of states, not the power of the machine.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	4
3	Tools and Technologies	5
4	Architecture Diagram	6
5	Methodology	8
6	Results	14
7	Conclusion	22
8	Reference	24

1. INTRODUCTION

1.1 Background

Almost every program that touches the web handles web addresses. A form field asks for a home-page link, a crawler reads thousands of links from a page, a firewall decides whether a link may be opened, and a chat application turns typed text into a clickable link. All of them first have to answer one small question: is this string a well-formed URL? The structure of a URL is defined by RFC 3986 [4]: a scheme such as `https`, then `://`, then a host, an optional port, and optionally a path, a query and a fragment. The host itself follows the naming rules of RFC 1035 and RFC 1123 [5]: labels separated by dots, each label made of letters, digits and hyphens, at most 63 characters long, with no hyphen at either end.

In the Theory of Computation, a set of strings that follows rules of this kind is a language, and the simplest class of languages is the class of regular languages. A regular language can be described by a regular expression, recognised by a non-deterministic finite automaton (NFA), and recognised by a deterministic finite automaton (DFA). The three descriptions are equivalent: this is the content of Kleene's theorem, of the early work of Rabin and Scott [6] and of the constructions in the textbooks of Hopcroft, Motwani and Ullman [1] and Sipser [2]. A DFA is the most practical of the three, because it reads the input once from left to right, moves between a fixed number of states, and uses a constant amount of work for each character.

The syntax of a URL, once the numeric limits are written out, is a regular language. This mini project turns that observation into software. The URL rules are written once as a single regular expression. A small compiler, written from the textbook algorithms, converts the expression into an NFA by Thompson's construction [3], the NFA into a DFA by the subset construction, and the DFA into the smallest equivalent DFA by partition refinement. Every URL typed by the user is then run through that minimal DFA. The result is a validator whose behaviour is exactly the language that the expression describes, and which can say not only whether a string is valid but where, and why, it stops being valid.

Table 1.1: Common ways of checking a URL and what each one gives

Approach	Strictness	Tells where it fails	Running time	Formal basis
Manual string checks (startswith, split, find)	Whatever the programmer remembers	Rarely	Linear	None
Library parser (`urllib.parse`)	Very lenient: splits almost any string	No	Linear	Splitting rules, not a grammar
Regular expression in a library (`re`)	Exactly the pattern	No: a failed match gives no reason	Usually linear; some patterns can be exponential [7]	Regular language, matched by backtracking
This project: minimal DFA	Exactly the grammar, including numeric ranges	Yes: position, character, expected characters	One table look-up per character, always linear	Regular language, decided by a DFA

1.2 Problem Statement

Programmers usually validate URLs in one of two unsatisfactory ways. Parsing libraries accept a great deal that is not a URL: for example `urllib.parse.urlparse` splits `http://exa mple.com` and `http://256.1.1.1` without complaint, so a program that trusts it lets malformed addresses through. Hand-written regular expressions are stricter, but they are written once, tested lightly, and then treated as a black box: when a match fails the only information is the word "no". Neither approach explains itself, and neither shows how the automata studied in the Theory of Computation decide a real language that students meet every day.

The problem addressed here is therefore to build a URL validator that (i) accepts exactly the strings of a precisely written URL grammar, (ii) is built by the automata constructions rather than by an off-the-shelf matcher, (iii) reports the position of the first error and the characters that would have been acceptable, and (iv) can be checked independently against another implementation of the same grammar.

1.3 Motivation

Three reasons motivated the project. First, URL validation is a problem every student recognises, which makes the abstract machinery of automata concrete. Second, the numeric rules of a URL are a good test of understanding: an IPv4 part must be at most 255 and a port at most 65535, and it is not obvious that a finite automaton, which has no counter, can check this. It can, because every number below a fixed bound is a finite set of strings, and Section 5.7 shows how the automaton does it digit by digit. Third, the construction pipeline has measurable properties (number of states before and after minimisation, running time) that can be reported and compared, which turns a classroom theorem into an experiment.

1.4 Scope

The validator covers absolute URLs with the schemes `http`, `https`, `ftp` and `ftps`, written in ASCII, with a host that is a domain name, an IPv4 address or the word `localhost`, an optional port from 1 to 65535, and an optional path, query and fragment made of the characters that RFC 3986 allows, including `%XX` escapes. It decides the syntax of a URL only. It does not contact the host, does not check that the top-level domain exists, and does not handle user names and passwords inside the address, IPv6 literals, internationalised names (these must be written in their `xn--` form), or relative references. These limits are listed again, with ways to remove them, in Section 7.

2. OBJECTIVES

The main objective is to design and implement a URL validator on the foundations of finite automata theory, and to evaluate it as software. The specific objectives are:

- To write the URL language as one regular expression assembled from named parts (scheme, host, port, path, query, fragment).
- To implement the complete pipeline from regular expression to minimal DFA: a regular-expression parser, Thompson's construction, the subset construction and Moore's minimisation, using only the Python standard library.
- To enforce the numeric limits (IPv4 parts up to 255, ports 1 to 65535, labels up to 63 characters) inside the automaton itself.
- To report the first error position, the offending character and the characters that would have kept the input valid, and to separate a wrong URL from an unfinished one.
- To provide a command-line interface and a browser interface, including a step-by-step trace of the automaton.
- To verify the validator with automated tests, an exhaustive check of the numeric ranges, and a comparison with Python's `re` module on thousands of generated strings.

Table 2.1: Functional requirements

ID	Requirement
FR1	Accept or reject a URL typed by the user, using the automaton built from the grammar
FR2	Distinguish three outcomes: valid, invalid (a character that cannot occur) and incomplete (input ended too early)
FR3	For an invalid URL, report the position and the character, and list characters that would have been accepted
FR4	For a valid URL, split it into scheme, host, host type, port, path, query and fragment
FR5	Show the state of the automaton after each character (trace)
FR6	Check many URLs at once, from a file or from the browser page
FR7	Show the size of the automata (NFA, DFA, minimal DFA)
FR8	Run from the command line and from a browser, with the same answers

Table 2.2: Non-functional requirements

Requirement	How it is met
Correctness	Same answers as an independent implementation (`re`) on 3,000 generated strings, plus exhaustive checks of numeric ranges
Speed	One table look-up per character; building the automaton takes under 0.1 s and is done once
Portability	Python standard library only; no packages to install
Explainability	Every rejection carries a position, a character, the expected characters and a plain-language hint
Maintainability	Grammar, automata engine, validator and interfaces are separate modules

3. TOOLS AND TECHNOLOGIES

The whole project is written in Python so that the code reads like the textbook constructions, and it deliberately uses no third-party package. Table 3.1 lists what was used.

Table 3.1: Tools and technologies used

Item	Tool	Purpose
Language	Python 3.10 or newer (developed and tested on 3.13)	All logic: parser, NFA, DFA, validator, interfaces
Libraries	Standard library only: `re`, `http.server`, `json`, `unittest`, `dataclasses`	`re` is used only as an independent check, never by the validator
Browser interface	One HTML file with CSS and plain JavaScript	Single check, batch check, trace table
Server	`http.server.ThreadingHTTPServer`	JSON endpoints used by the page
Testing	`unittest`	21 automated tests
Measurement	`time.perf_counter`, matplotlib (charts for this report only)	Automaton sizes and running times
Editor and shell	Visual Studio Code, PowerShell or any terminal	Development

3.1 Running the project

The project is a folder named `urlvalidator`. No installation step is needed. From inside the folder:

```
python -m unittest discover -s tests        # run the 21 automated tests
python -m urlvalidator.cli https://example.com/a?b=1 --trace
python -m urlvalidator.cli --info           # sizes of the automata
python -m urlvalidator.cli --file urls.txt  # one URL per line
python -m urlvalidator.server 8000          # browser interface at
http://127.0.0.1:8000
python bench.py                             # measurements used in Chapter 6
```

The command-line program returns exit code 0 when every URL given is valid and 1 otherwise, so it can be used in scripts.

4. ARCHITECTURE DIAGRAM

The validator has four layers (Figure 4.1). The presentation layer is either the command line or the browser page, which talks to a small HTTP server. The application layer holds the validator itself: it runs the automaton, builds the error report and splits the components of a valid URL. The automata engine contains the algorithms of the Theory of Computation: the regular-expression parser, Thompson's construction, the subset construction, Moore's minimisation and the DFA runner. At the bottom, the grammar module holds the URL language as one regular expression. A layer calls only the layer below it, so the engine can be tested without any interface.

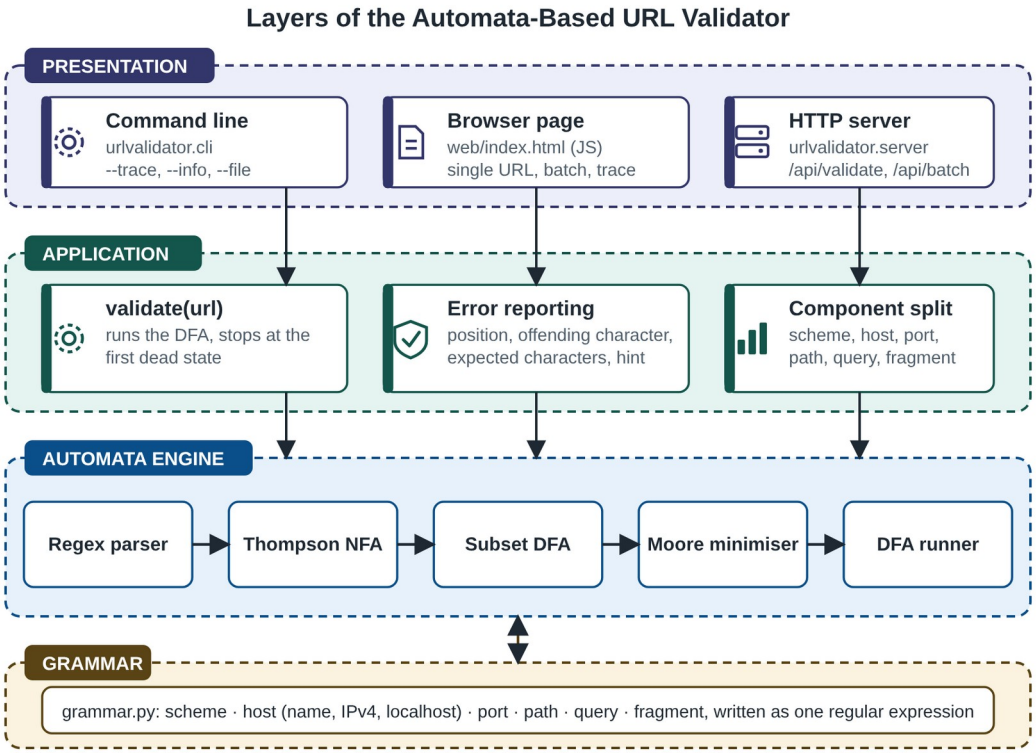


Figure 4.1: Layers of the Automata-Based URL Validator

Table 4.1: Module responsibilities

File	Lines	Responsibility
`urlvalidator/automata.py`	313	Regex parser, Thompson NFA, subset construction, Moore minimisation, DFA runner, equivalence test
`urlvalidator/grammar.py`	29	The URL language as named regular-expression pieces
`urlvalidator/validator.py`	113	`validate()`, error reporting, phase labels, component split
`urlvalidator/cli.py`	36	Command-line interface
`urlvalidator/server.py`	52	HTTP server and JSON endpoints
`urlvalidator/cases.py`	39	36 labelled demonstration cases
`web/index.html`	115	Browser page
`tests/test_urlvalidator.py`	153	21 automated tests
`bench.py`	66	Measurements

4.1 Building once, running for every URL

Figure 4.2 separates the work that is done once from the work done for every URL. At start-up, the grammar is compiled to an NFA of 618 states, a DFA of 299 states and a minimal DFA of 258 states, which takes about 59 ms. The minimal DFA is stored as a table with one row per state and one column per character class (29 classes, because the 128 ASCII characters fall into only 28 groups that the grammar can tell apart, and every non-ASCII character is one more class). For every URL the validator reads one character at a time, looks up the next state, and stops as soon as the state can no longer reach an accepting state. This dead state is detected in constant time from a set computed once, by a backward search from the accepting states.

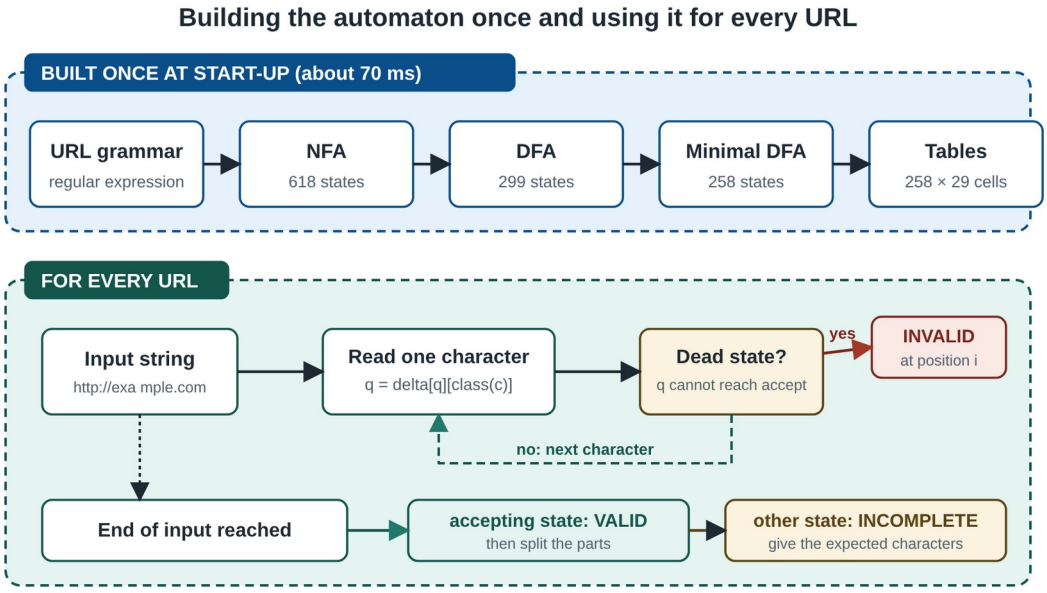


Figure 4.2: Building the automaton once and using it for every URL

5. METHODOLOGY

5.1 Overall approach

The method follows the classical chain of equivalences: regular expression, NFA, DFA, minimal DFA. Each step is an algorithm that is implemented in `automata.py` and tested on its own. The grammar of URLs is then fed to the chain, and the resulting machine is wrapped by the validator. The decision to use the constructions directly, instead of calling a library, is what makes every number reported in Chapter 6 (states before and after minimisation, for example) a measurement of the algorithms rather than of somebody else's code.

5.2 The URL grammar

The language is written as one regular expression assembled from named pieces, shown in Table 5.1. The same text is valid in Python's `re` module, which is what allows the independent check in Section 6.5. Case-insensitive words such as `http` are written as classes (`[hH][tT][tT][pP]`), so the automaton needs no special notion of case.

Table 5.1: The URL language as named regular expressions

Part	Regular expression	Meaning
scheme	`(https?|ftps?)`, case-insensitive	http, https, ftp, ftps
octet	`(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])`	0 to 255, no leading zeros
IPv4	`octet\.octet\.octet\.octet`	four parts
label	`[A-Za-z0-9]([A-Za-z0-9-]{0,61}[A-Za-z0-9])?`	1 to 63 characters, no hyphen at either end
domain	`(label\.)+[A-Za-z]{2,63}`	labels, then a top-level label of 2 to 63 letters
host	`(localhost|IPv4|domain)`	one of the three
port	`(:(6553[0-5]|655[0-2][0-9]|65[0-4][0-9]{2}| 6[0-4][0-9]{3}|[1-5][0-9]{4}|[1-9][0-9] {0,3}))?`	optional, 1 to 65535
pchar	`([A-Za-z0-9\-._~!$&'()*+,;=:@]|%[0-9A-Fa-f] {2})`	path character
path	`(/pchar*)*`	zero or more segments
query	`(\?(pchar|[/?])*)?`	optional
fragment	`(#(pchar|[/?])*)?`	optional
URL	`scheme://host port path query fragment`	the whole language

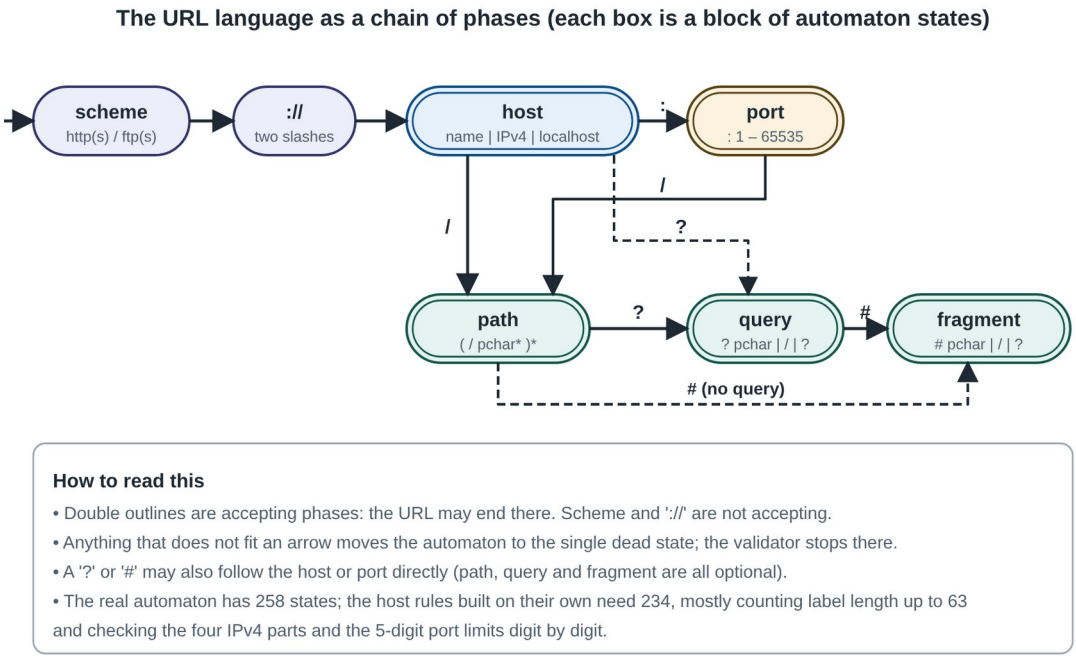


Figure 5.1: The URL language as a chain of phases

5.3 Parsing the regular expression

The expression is read by a recursive-descent parser for the usual grammar: an alternation is a list of concatenations separated by `|`, a concatenation is a list of repeated atoms, and an atom is a group, a character class, an escaped character or a literal. The repetition operators are `*`, `+`, `?`, `{m}` and `{m,n}`. The parser returns a syntax tree whose nodes are: empty, character set, concatenation, alternation, star, plus, optional and counted repetition. Unbalanced brackets and a repetition with nothing to repeat raise an error, which a test checks.

5.4 Thompson's construction

Each node of the tree becomes a small NFA with one start state and one accepting state, joined to its neighbours by empty (epsilon) transitions, exactly as in Thompson's construction [3] and in the compiler textbooks [9]. A character set becomes one transition labelled with the set; a concatenation links the accepting state of each part to the start of the next; an alternation adds a new start and a new end with epsilon edges to every branch; a star adds the two loop edges. A counted repetition `x{m,n}` is expanded into `m` copies of `x` followed by `n - m` optional copies, which is why bounded labels make the NFA larger. The full URL expression produces an NFA with 618 states.

5.5 Subset construction

The NFA is converted to a DFA by the subset construction. Each DFA state stands for the set of NFA states that the NFA could be in after reading the same input. The start state is the epsilon-closure of the NFA start state. For every symbol, the next DFA state is the closure of all NFA states reachable by that symbol from any state in the current set. New sets are added to a queue until no new set appears. A DFA state is accepting when its set contains the NFA accepting state, and the empty set is the dead state.

Two implementation details keep the result small. First, the alphabet is compressed. The sets used in the pattern divide the 128 ASCII characters into groups that no set can tell apart; for the URL grammar there are 28 such groups, plus one class for all non-ASCII characters. The construction works on classes, not on single characters, which divides the work by about four. Second, non-ASCII characters always lead to the dead state, so an address such as `http://münchen.de` is rejected at the character ü.

The classic example of the textbooks is `(a|b)*abb`, the strings of a's and b's that end in abb. Its NFA has 16 states, the DFA made by the subset construction has 6, and the minimal DFA has 5: the four states of the textbook answer and one dead state. The same code runs on the URL grammar with no change.

5.6 Minimisation

The DFA from the subset construction may contain states that behave identically. Moore's algorithm [8] merges them. It starts with two blocks, accepting and non-accepting states, and repeatedly splits a block when two of its states go to different blocks on some symbol, until no block can be split. The remaining blocks are the states of the minimal DFA, which has the fewest states of any DFA for the language. The core of the implementation is:

```
block = [1 if s in dfa.accepting else 0 for s in range(n)]
while True:
    sig, new = {}, []
    for s in range(n):
        # a state is described by its own block and the blocks it leads to
        key = (block[s], tuple(block[t] for t in dfa.delta[s]))
        new.append(sig.setdefault(key, len(sig)))
    if len(sig) == len(set(block)):
        break                       # no block was split: finished
    block = new
```

For `(a|b)*abb` the minimal DFA has the transition table of Table 5.2. For the URL grammar the minimisation removes 41 of the 299 states. Two tests confirm the result: the minimal DFA and the original DFA are proved equivalent by the product construction (Section 5.10), and minimising the minimal DFA again changes nothing.

Table 5.2: Minimal DFA for (a|b)*abb (state 4 accepts; state 1 is the dead state because the other character classes lead there)

State	on a	on b	any other character
0 (start)	2	0	1
1 (dead)	1	1	1
2	2	3	1
3	2	4	1
4 (accepting)	2	0	1

5.7 Numbers and counts in a language without counters

A finite automaton cannot count without limit, but every limit used in a URL is finite, so every limited number is a regular language. An IPv4 part from 0 to 255 is written as the alternation `25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9]`: after a `2` the automaton remembers whether a `5` has been read, and so on. A port from 1 to 65535 needs the six alternatives of Table 5.1. A label of at most 63 characters is `{0,61}` between two edge characters, which the construction expands into 61 optional copies. The cost of these rules is the number of states: the minimal DFA for an octet has 7 states, for a port 12, and for a label 126. Section 6.8 shows that the bound on the label length is the main reason why the whole URL automaton has 258 states rather than a few dozen.

The checks are exhaustive and do not rely on samples: the tests feed every integer from 0 to 999 to the octet automaton and every integer from 0 to 69,999 to the port automaton, and compare the answer with the plain arithmetic condition. Leading zeros such as `01` and `007` are rejected by design.

5.8 Running the automaton and reporting errors

Running a DFA is a loop with one table look-up per character:

```
def accepts(self, text):
    d, q, cc, other = self.delta, self.start, self.char_class, self.other
    for ch in text:
        o = ord(ch)
        q = d[q][cc[o] if o < 128 else other]
    return q in self.accepting
```

The validator uses the same loop but stops early. Before the loop, a backward search from the accepting states marks every *live* state, one from which an accepting state is still reachable. The states that are not live form the dead region; in the URL automaton it is a single sink state. If the next state is not live, the string can no longer become a URL whatever follows, so the position of the character just read is reported. The characters that were acceptable are found by trying each printable ASCII character from the previous state

and keeping those whose target is live. If the loop ends in a state that is live but not accepting, the URL is **incomplete** and the expected characters are listed in the same way. This gives three outcomes (valid, invalid, incomplete), which a plain yes-or-no matcher cannot give.

A plain-language hint is added from the part of the URL in which the error occurred. The part is found by a simple scan of the text read so far: after `://` the host continues until a `:`, `/`, `?` or #, and the latest of these delimiters decides whether the position is in the port, path, query or fragment. The hint never changes the verdict; the verdict always comes from the automaton. One exception improves the message only: when a dotted host has four numeric parts and one is above 255, such as `256.1.1.1`, the message names the part, because the automaton alone would blame the first character after the host.

5.9 Splitting a valid URL

For a URL that the automaton has accepted, the components are found by cutting at the first #, then the first `?`, then the first `/` after the host. This is exact because none of these characters can occur inside the part before it. The host is classified as `localhost`, `ipv4` or `domain`, and the scheme is reported in lower case.

5.10 Checking two automata for equivalence

Two DFAs accept the same language exactly when no pair of states (p, q), reachable together from the two start states, has one accepting and one rejecting member. The `equivalent` function explores the pairs by breadth-first search over all 128 ASCII characters and one non-ASCII character. It is used in the tests to confirm that the subset DFA and the minimal DFA accept the same language, and to confirm that two different expressions, such as `(a|b)*` and `(a*b*)*`, describe the same language while `a*` and `a+` do not.

Table 5.3: Cost of the procedures (m = length of the pattern, n = NFA states, k = character classes, L = length of the URL)

Procedure	Time	Remark
Parse the expression	O(m)	recursive descent
Thompson's construction	O(m)	at most two states per node; counted repetition multiplies the copies
Subset construction	O(2^n · k) worst case	measured: 618 NFA states give only 299 DFA states
Moore's minimisation	$O(s^2 \cdot k)$ worst case	s = DFA states; a few rounds in practice
Running the DFA	O(L)	one look-up per character, independent of the grammar size
Equivalence test	$O(s_1 \cdot s_2 \cdot k)$	pairs of states

5.11 Testing methodology

Four kinds of test were used. **Unit tests** check each stage: the parser, the construction on the textbook example, equivalence, and the behaviour on non-ASCII text. **Exhaustive tests** check the numeric rules for every integer in a range and the label limit at 63 and 64 characters. **Labelled cases** are 36 hand-written URLs, 12 valid and 24 invalid, each with a reason (Table 6.2). **Differential testing** generates 3,000 strings by deleting, inserting and replacing characters in valid URLs and requires the validator and `re.fullmatch` (given the identical pattern) to agree on every one. The two interfaces are tested by starting the server on a free port, requesting the JSON endpoints and checking the replies.

6. RESULTS

6.1 Test results

All 21 automated tests pass (`python -m unittest discover -s tests`, run time under one second). Table 6.1 lists them by group.

Table 6.1: Automated tests

Group	Tests	What is checked
Automata	7	textbook example (4 live states + sink); subset DFA ≡ minimal DFA; minimising twice changes nothing; malformed patterns rejected; counted repetition; non-ASCII never accepted; equivalence detects a difference
Numeric ranges	3	octet accepts exactly 0 to 255 (all integers 0 to 999); port exactly 1 to 65535 (all integers 0 to 69,999); label limit 63 versus 64
Validator	8	all 36 labelled cases; error position and expected characters; incomplete input; bad IPv4 message; component split; trace length; phase labels; empty string
Differential	1	3,000 mutated URLs, validator = `re.fullmatch` on every one
Interfaces	2	command-line exit codes; server JSON for /api/validate, /api/info and /api/batch

6.2 Demonstration cases

Table 6.2 shows the 36 labelled cases and the answer of the validator. The validator agrees with the label in all 36 of 36 cases. For invalid URLs the table gives the position of the first error, which is the index of the first character that cannot belong to any URL with that beginning; for incomplete ones it is the length of the input.

Table 6.2: Demonstration cases and validator output

#	URL	Result	Pos.	Reason
1	`https://example.com`	valid		plain https URL
2	`http://www.rmkcet.ac.in/index.html`	valid		sub-domain, path
3	`http://localhost:8080/api/v1/users?id=42#top`	valid		localhost, port, path, query, fragment
4	`ftp://files.example.org/pub/readme.txt`	valid		ftp scheme
5	`HTTPS://EXAMPLE.COM/Path`	valid		scheme and host are case-insensitive
6	`http://192.168.0.1/admin`	valid		IPv4 host
7	`http://255.255.255.255:65535/`	valid		largest IPv4 and port
8	`https://a-b.example.co.uk/%E0%A4%A4?q=a%20b`	valid		hyphen in label, percent escapes, 3-level domain
9	`http://example.com/a/b/c/`	valid		trailing slash
10	`http://example.com?search=url#frag/ment?x`	valid		query and fragment without a path
11	`http://1.2.3.4.example.com`	valid		digit-only labels before a letter TLD
12	`http://example.com:1`	valid		smallest port
13	`example.com`	invalid	0	no scheme
14	`http:/example.com`	invalid	6	only one slash after the colon
15	`htp://example.com`	invalid	2	misspelled scheme

#	URL	Result	Pos.	Reason
16	`mailto://example.com`	invalid	0	scheme not supported
17	`http://`	incomplete	7	host missing
18	`http://exa mple.com`	invalid	10	space inside the host
19	`http://-example.com`	invalid	7	label starts with a hyphen
20	`http://example-.com`	invalid	15	label ends with a hyphen
21	`http://exa_mple.com`	invalid	10	underscore in a host name
22	`http://example`	incomplete	14	single label that is not localhost
23	`http://example.c`	incomplete	16	top-level label shorter than two letters
24	`http://example.123`	incomplete	18	top-level label is not alphabetic
25	`http://256.1.1.1`	invalid	7	IPv4 part above 255
26	`http://1.2.3`	incomplete	12	IPv4 address with three parts
27	`http://example.com:0`	invalid	19	port zero
28	`http://example.com:65536`	invalid	23	port above 65535
29	`http://example.com:`	incomplete	19	colon without a port
30	`http://example.com:80a/`	invalid	21	letter inside the port
31	`http://example.com/a b`	invalid	20	space in the path
32	`http://example.com/%zz`	invalid	20	bad percent escape
33	`http://example.com/a<b>`	invalid	20	angle brackets in the path
34	`http://example.com/?q=a b`	invalid	23	space in the query
35	`http://münchen.de`	invalid	8	non-ASCII host (needs punycode)
36	`http://aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa...`	invalid	70	label of 64 characters

6.3 Worked example: the automaton on a URL

Table 6.3 follows the minimal DFA through the valid URL `http://a.io/x?y=1`. The state numbers are those of the machine built by the program. The path, query and fragment characters keep the automaton in one state, because after the host these parts accept the same characters and the minimal DFA does not distinguish them (the `?` and # markers only matter for the later split). The phase column is the label added by the validator.

Table 6.3: DFA trace for http://a.io/x?y=1

#	Char	State	Phase
0	`h`	q3	scheme
1	`t`	q2	scheme
2	`t`	q4	scheme
3	`p`	q5	scheme
4	`:`	q6	scheme
5	`/`	q8	scheme
6	`/`	q9	host
7	`a`	q14	host
8	`.`	q22	host
9	`i`	q30	host
10	`o`	q39	host
11	`/`	q49	path
12	`x`	q49	path
13	`?`	q49	query
14	`y`	q49	query
15	`=`	q49	query
16	`1`	q49	query

The run ends in state q49, which is accepting, so the URL is valid. The same program on `http://exa mple.com` stops at index 10, the space, because from the state reached after `exa` no transition on a space leads to a live state. It reports that the host allows letters, digits, hyphens and dots, and lists the characters that would have been accepted.

6.4 Size of the automata

Table 6.4 and Figure 6.1 give the number of states at each stage, for each part of the grammar and for the whole URL. The subset construction turns 618 NFA states into 299 DFA states, and minimisation reduces them to 258. The minimal automaton has 5,270 transitions between live states over 29 character classes. Pieces that only repeat a few characters, such as the path, query and fragment, need only 5 states each. The host, with its bounded labels and the four-part IPv4 address, is the largest part, and the label rule alone needs 126 states.

Table 6.4: Number of states at each stage of the pipeline

Grammar part	NFA	DFA	Minimal DFA	Classes	Build (ms)
scheme	28	11	7	7	0.3
ipv4	144	45	25	9	1.5
label	134	126	126	4	5.8
domain	268	190	190	6	13.0
host	434	246	234	18	30.0
port	80	26	12	10	0.8
path	20	7	5	6	0.2
query	24	8	5	7	0.3
fragment	24	8	5	7	0.2
full URL	618	299	258	29	57.5

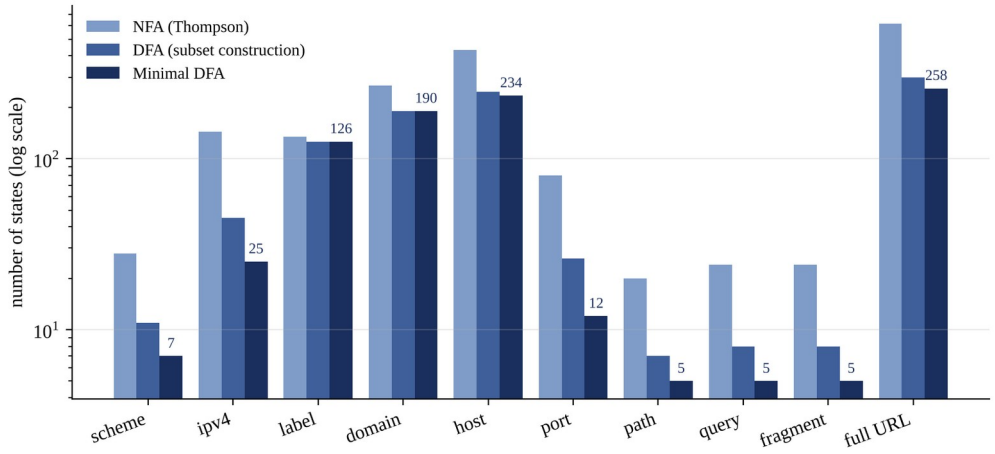


Figure 6.1: States at each stage of the pipeline, by grammar part

6.5 Agreement with Python's re module

The strongest check of correctness is independent agreement. Python's `re` module uses a different algorithm (backtracking) and different code, but it can be given the identical pattern string. The test generates 3,000 strings by applying up to three random edits (delete, insert, replace, using characters from a set chosen to include URL delimiters, a space and a non-ASCII letter) to the valid demonstration URLs. The validator and `re.fullmatch` gave the same answer on all 3,000, and also on all 36 labelled cases. The numeric ranges were checked exhaustively as described in Section 5.7. This does not prove that the grammar is what RFC 3986 says, only that the automaton implements the written grammar exactly; the grammar itself is checked by the labelled cases.

6.6 Comparison with a parsing library

As a baseline, a typical library check was applied to the same 36 cases: `urlparse(url)` followed by the test "the result has a scheme and a network location". Table 6.5 gives the accuracy. The library agrees with the labels in only 15 of 36 cases because it accepts 21 of the 24 invalid URLs, including a misspelled scheme, a space in the host, an IPv4 part of 256, port 0 and port 65536, and a non-ASCII host. It rejects none of the valid ones. This is not a defect of the library, which is designed to split text rather than to validate it, but it shows why splitting is not validation.

Table 6.5: Accuracy on the labelled cases

Method	Correct (of 36)	Invalid URLs wrongly accepted
Minimal DFA (this project)	36	0
`re.fullmatch` with the same pattern	36	0
`urlparse` (scheme and host present)	15	21

6.7 Browser interface

Figures 6.2 to 6.5 are screenshots of the browser interface (`python -m urlvalidator.server`). The header states that every URL is run through a minimal DFA, and the footer reports the size of the automaton built from the grammar (618 NFA states, 299 DFA states, 258 after minimisation, 29 character classes).

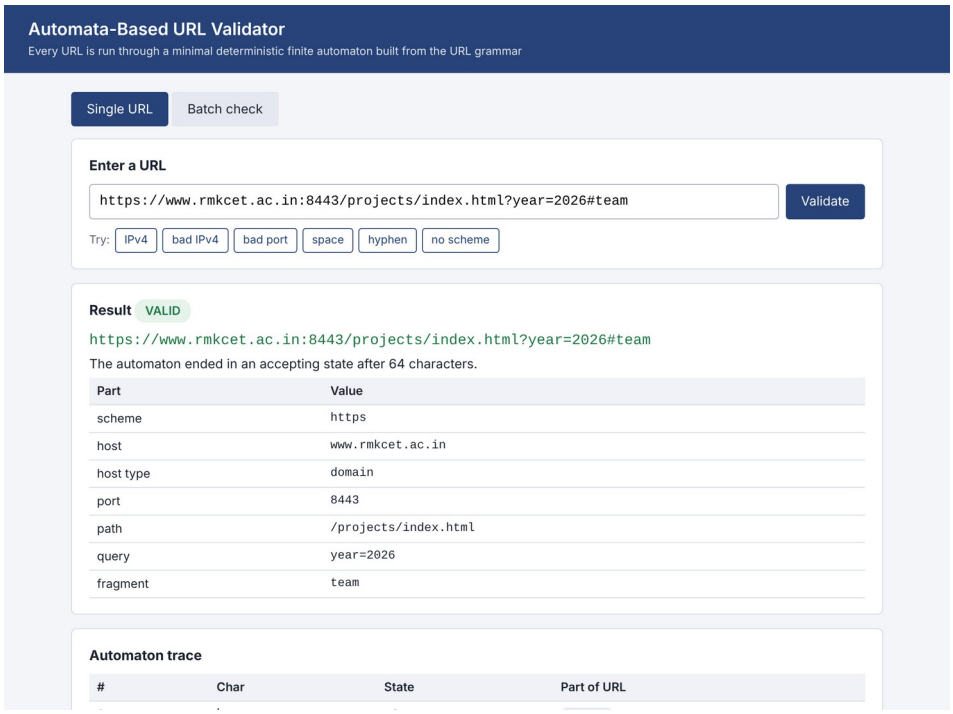


Figure 6.2: A valid URL: verdict, components and the first rows of the trace

Figure 6.2 shows an address that is accepted: `https://www.rmkcet.ac.in:8443/projects/index.html?year=2026#team`. The result card shows the URL in green and a table with the scheme, host, host type (domain), port 8443, path, query and fragment. Below it the trace lists the state reached after each character together with the part of the URL the character belongs to. The row of buttons above the result loads examples that illustrate each kind of error.

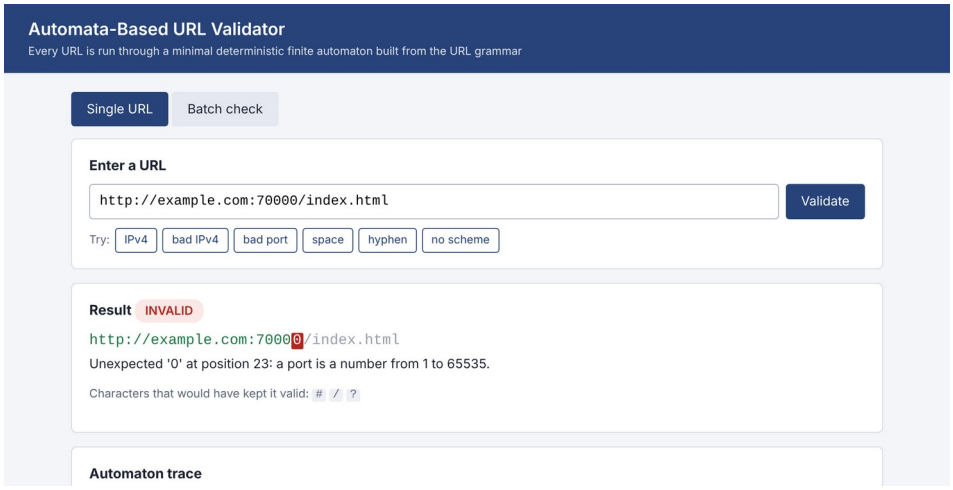


Figure 6.3: Port above 65535: rejected at the fifth digit

Figure 6.3 shows `http://example.com:70000/index.html`. The first four digits of the port are accepted, because `7000` is a valid port and the number can still grow, but the fifth digit makes 70000, which is above 65535, so the automaton enters the dead state. The URL is shown with the offending character highlighted in red, the message says that a port

is a number from 1 to 65535, and the digits that would have been accepted at that point are listed.

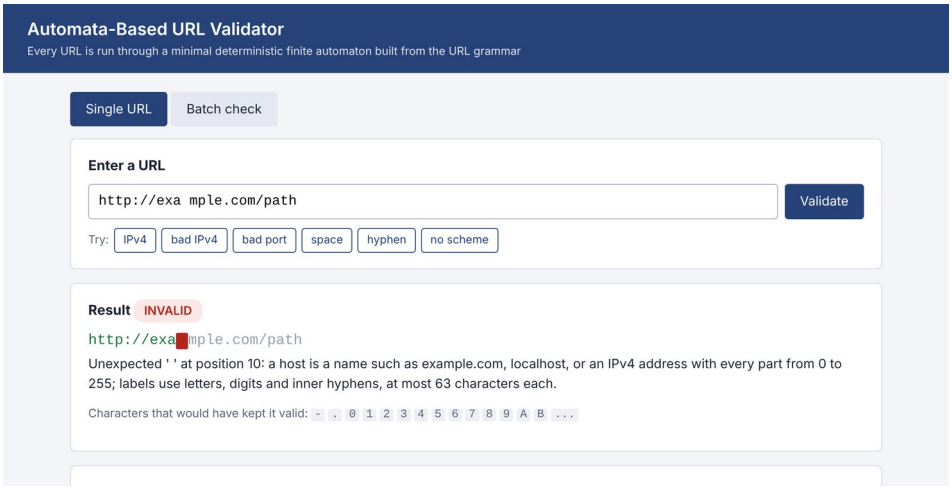


Figure 6.4: A space in the host name

Figure 6.4 shows `http://exa mple.com/path`. The green part is the valid prefix, the red block is the space at position 10, and the grey remainder is the text after the error, which the automaton never reads. The expected characters shown are the hyphen, the dot, digits and letters, which are all that a host name may contain at this point.

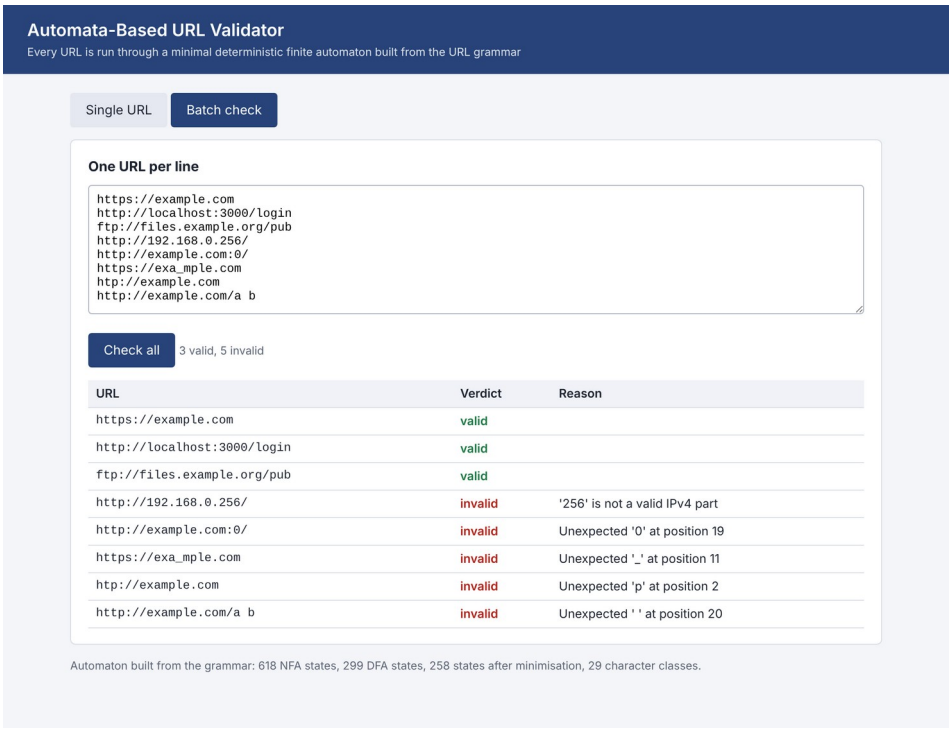


Figure 6.5: Batch check of eight URLs

Figure 6.5 is the batch page. Eight URLs, one per line, are sent in one request and the page reports three valid and five invalid with the first words of the reason: the IPv4 part 256,

port 0, the underscore in the host name, the misspelled scheme at position 2 and the space in the path at position 20.

6.8 Performance

Running time was measured on the 36 labelled cases repeated 200 times (7,200 URLs), taking the best of three runs. The measurements are from one laptop-class machine and one Python version, so they show the relative size of the numbers, not an absolute speed.

Table 6.6: Throughput on short URLs (URLs per second)

Method	URLs per second	Remark
Bare DFA loop (`accepts`)	757,468	yes or no only
`re.fullmatch`, same pattern	819,981	C implementation, yes or no only
`urlparse` and two tests	1,026,241	splits only, does not validate
`validate()` with full report	74,620	builds position, message, expected characters, components

The bare automaton runs at about the same speed as the compiled `re` module [10], which is a good result for a loop written in Python. The full `validate()` is about 10 times slower because it also builds the explanation for every URL: it scans the text for phases, computes the expected characters and splits the components. At 74,620 URLs per second this is still far more than a form or a page needs. Figure 6.6 shows that the running time grows in a straight line with the length of the URL, about 48 nanoseconds per character, as the theory predicts: 100,000 characters take 4.8 ms.

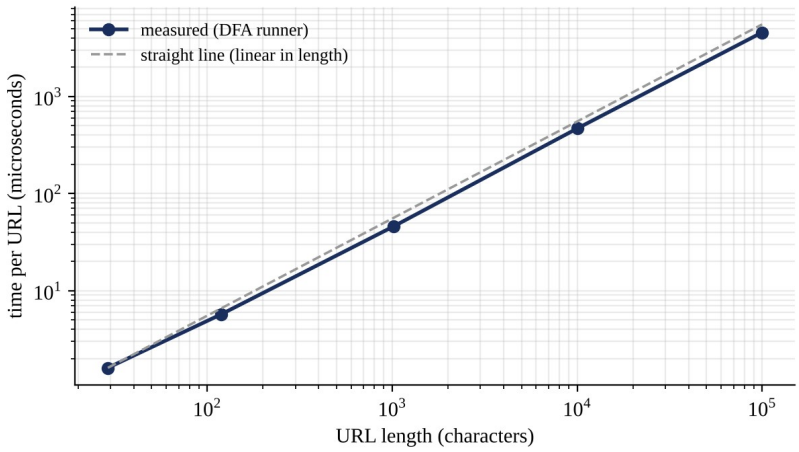


Figure 6.6: Running time against URL length (both axes logarithmic)

The price of a bounded label is the size of the automaton. Figure 6.7 rebuilds the whole URL automaton with the longest allowed label set to 2, 8, 16, 32 and 63 characters. The minimal DFA grows from 88 states to 258, roughly three more states for every extra character of label length, because the automaton has to remember how long the current

label is. The growth is linear, not exponential, and the machine stays small: all 258 states and 29 columns fit in a table of about 7,482 cells. Building takes about 59 ms and is done once.

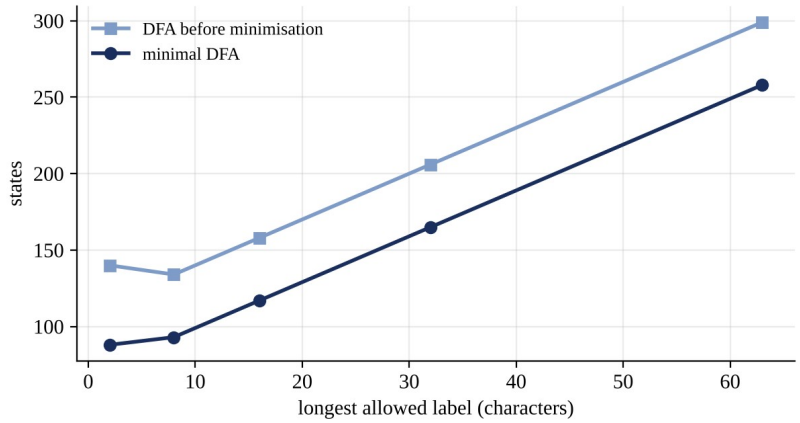


Figure 6.7: States of the URL automaton against the longest allowed label

6.9 Discussion

The results support the three claims of the project. First, the construction chain works on a real grammar: 618 NFA states become 258 minimal DFA states, and the minimal automaton agrees with an independent matcher on every generated string. Second, a finite automaton can enforce numeric limits, since every limit used in a URL is finite; the price is states, not a more powerful machine. Third, the DFA gives information that a yes-or-no matcher cannot: the position of the error, the characters that would have been valid, and the distinction between a wrong URL and an unfinished one.

The results also show where the approach is weaker. A bound that is large (a path limited to 2,000 characters) would need thousands of states, which is why the grammar does not limit the length of the path or query. Facts that live outside the string, such as whether the top-level domain exists or whether the host answers, cannot be decided by an automaton at all. And the automaton is only as correct as the grammar it is built from: the 36 labelled cases were written by the authors, and a wider set of real-world addresses would be needed to find rules that were forgotten.

7. CONCLUSION

This mini project built an automata-based URL validator from first principles. The URL syntax was written as one regular expression, converted to an NFA by Thompson's construction, to a DFA by the subset construction and to a minimal DFA by Moore's algorithm, giving a machine of 258 states that reads each URL in a single pass. The numeric rules of a URL (IPv4 parts up to 255, ports 1 to 65535, labels up to 63 characters) are enforced inside the automaton, and the machine reports the position of the first error, the characters that would have been valid, and whether the input is wrong or only unfinished. The validator agrees with all 36 labelled cases, with 3,000 generated strings checked against Python's `re`, and with exhaustive checks of the numeric ranges; 21 automated tests pass. A command-line interface and a browser interface with a state-by-state trace make the automaton visible.

The project also gave practical lessons about the Theory of Computation: that a language with arithmetic-looking rules can still be regular, that the size of the machine rather than its power is the cost of precision, and that minimisation matters (258 states instead of 299).

7.1 Limitations

- The validator checks syntax only. It does not check that a domain exists or that a server answers.
- The top-level label is any two to 63 letters, so `example.zzz` is accepted although `.zzz` is not a real top-level domain.
- User names and passwords in the address, IPv6 literals, internationalised names written in Unicode and relative references are outside the grammar.
- The set of labelled cases was written by the authors and is small; the agreement with `re` shows that the automaton implements the grammar, not that the grammar is complete.
- Timings come from one machine and one Python version.
- The grammar bounds the length of a host label but not of the path, query or fragment.

7.2 Future scope

- Accept IPv6 literals (`[2001:db8::1]`) and a user-information part, which only change the grammar.
- Accept internationalised names by converting them to the `xn--` form before the automaton runs.

- Add a list of real top-level domains as a second automaton (a finite set of strings is regular) and update it from the public list.
- Evaluate the validator on a large collection of real-world links and compare the disagreements with other validators.
- Generate the automaton as a C or JavaScript table so that the same machine can run in a browser form field without a server.
- Draw the state diagram of any pattern, which would make the tool a teaching aid for NFA, DFA and minimisation.
- Add counters for limits that are too large for states, such as the 255-character total length of a host name, as a second stage after the automaton.

8. REFERENCE

[1] J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed. Boston, MA: Pearson, 2006.

[2] M. Sipser, Introduction to the Theory of Computation, 3rd ed. Boston, MA: Cengage Learning, 2012.

[3] K. Thompson, "Programming techniques: Regular expression search algorithm," Communications of the ACM, vol. 11, no. 6, pp. 419–422, 1968.

[4] T. Berners-Lee, R. Fielding and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," RFC 3986, Internet Engineering Task Force, 2005.

[5] P. Mockapetris, "Domain names - implementation and specification," RFC 1035, 1987; and R. Braden, Ed., "Requirements for Internet Hosts - Application and Support," RFC 1123, 1989.

[6] M. O. Rabin and D. Scott, "Finite automata and their decision problems," IBM Journal of Research and Development, vol. 3, no. 2, pp. 114–125, 1959.

[7] R. Cox, "Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...)," 2007. [Online]. Available: https://swtch.com/~rsc/regexp/regexp1.html

[8] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton University Press, 1956, pp. 129–153.

[9] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[10] Python Software Foundation, "re - Regular expression operations" and "urllib.parse - Parse URLs into components," Python 3 documentation. [Online]. Available: https://docs.python.org/3/